Executive Summary — Freelance Marketplace DApp

Programming Project, Option A (non-tutorial) • Built from scratch (no seed DApp used)

Solidity 0.8.20 • Truffle 5.11.5 • Ganache 7.9.1 • Python 3.12.7 / Flask 3.0.3 / web3.py 6.20.4 • HTML/CSS/JS • web3.js 1.10.0 • MetaMask

What the DApp is. A decentralized freelance marketplace on Ethereum where employers post jobs with milestone-based escrow, freelancers bid, and ETH is released milestone-by-milestone as work is completed. The system is non-custodial: the Flask backend never holds private keys — every state-changing call is signed by the user's own wallet in MetaMask. The smart contract is the single source of truth; Flask serves only as a read, index, and config layer.

What it does. The employer calls `postJob`, depositing the exact sum of milestone amounts into escrow (`require(msg.value == sum(milestoneAmounts))`). Freelancers submit bids with an amount, proposal, and expiry duration. The employer accepts one bid (Open → InProgress), the freelancer marks milestones complete, and the employer releases payment per milestone under Checks-Effects-Interactions with OpenZeppelin `ReentrancyGuard`. When all milestones are paid the job auto-completes, and the employer can rate the freelancer on-chain (1–5). If the parties disagree, either can raise a dispute, freezing all payments. A separate `MultiSigArbitration` contract runs a 2-of-3 arbitrator vote; if no consensus within 7 days, either party can trigger a deterministic 50/50 split. The contract is `Pausable` by the owner for emergencies.

Why it is relevant. Centralized platforms (Upwork, Fiverr) extract 10–20% fees, custody funds, control dispute outcomes unilaterally, and own reputation data workers cannot port. This DApp removes the middleman: escrow is enforced in code, dispute resolution is transparent multisig with a timeout fallback, and reputation lives on-chain tied to a wallet address — portable, tamper-proof, and auditable.

The 6 major features (each involves changes to both Solidity and the frontend):

1. **Multi-milestone escrow.** *Solidity:* `Milestone` struct with per-milestone `completed`/`paid` flags, CEI + `nonReentrant` on all payment release functions, deposit invariant enforced in `postJob`, auto-transition to Completed on last payment. *Frontend:* dynamic milestone form, visual progress tracker, per-milestone release buttons, BigInt wei math.
2. **Bid expiry system.** *Solidity:* `expiresAt` field on bids, expiry enforced in `acceptBid`, `withdrawExpiredBid` callable by the freelancer after expiry. *Frontend:* duration input, countdown timer on bid cards, accept button auto-disabled after expiry.
3. **2-of-3 multisig dispute resolution with 7-day timeout.** *Solidity:* separate `MultiSigArbitration.sol` contract, vote tallying with consensus check, `claimTimeout` 50/50 split after 7 days, replaceable arbitrators. *Frontend:* arbitrator dashboard with vote UI, dispute button on job detail, Flask `/api/disputes` endpoint.
4. **On-chain reputation and ratings.** *Solidity:* `rateFreelancer` with one-time-per-job gate, cumulative `reputationScore` and `ratingCount` on `Profile`, `totalJobsCompleted` auto-incremented. *Frontend:* star rating widget on completed jobs, average rating on profile cards and bid listings.
5. **User profiles and earnings dashboard.** *Solidity:* `Profile` struct with name, bio, job counters, create/update events. *Frontend:* profile forms in both dashboards, live freelancer earnings summary (Total Earned / Pending Release / Locked in Escrow) computed from on-chain milestone data.
6. **Category filter and search.** *Solidity:* `category` field on `Job`, indexed in `JobPosted` events. *Backend:* Flask event indexer with persisted block cursor, `/api/jobs/category/<cat>`. *Frontend:* category filter bar and keyword search on the job board.

Security and testing. CEI ordering and OpenZeppelin `ReentrancyGuard` on all payment paths; `Pausable` by owner for emergency freeze. A 76-test pytest suite covers escrow invariants, authorization, dispute edge cases, event correctness, state-machine transitions, and timeout handling — all passing against a fresh Ganache deployment.